Oracle DB: User Management Notes

Covers creating users, listing users, switching users, changing passwords, granting privileges, privilege types, and roles.

1. Before You Start

- Connect as a user who has `CREATE USER` privilege: `SYS`, `SYSTEM`, or a DBA.
- **Multitenant (12c and later):** in the root container (`CDB$ROOT`) you can only create *common users*, which must start with `C##`. To create a normal *local user*, switch to a pluggable database first.

```
-- Check where you are
SHOW CON_NAME;

-- Switch to the pluggable database (name varies; XE usually uses XEPDB1)
ALTER SESSION SET CONTAINER = XEPDB1;
```

2. Create a User

Basic syntax

```
CREATE USER username IDENTIFIED BY password;
```

Complete example

```
CREATE USER aasif IDENTIFIED BY "StrongPass#123"
  DEFAULT TABLESPACE users
  TEMPORARY TABLESPACE temp
  QUOTA UNLIMITED ON users;
```

Clause	Purpose
`IDENTIFIED BY`	Sets the password
`DEFAULT TABLESPACE`	Where the user's tables and indexes are stored
`TEMPORARY TABLESPACE`	Used for sorting and temporary work

Clause	Purpose
`QUOTA ... ON ...`	Space the user may use. Without a quota, the user cannot store table data
`PASSWORD EXPIRE`	Forces a password change at first login
`ACCOUNT LOCK` / `ACCOUNT UNLOCK`	Locks or unlocks the account

A new user **cannot even log in** until given `CREATE SESSION`:

```
GRANT CREATE SESSION TO aasif;
```

Drop a user

```
DROP USER aasif;           -- fails if the user owns objects
DROP USER aasif CASCADE;   -- also deletes all of the user's objects
```

3. Show Users

Current user

```
SHOW USER;                       -- SQL*Plus / SQLcl command
SELECT USER FROM DUAL;           -- works in any SQL tool
SELECT SYS_CONTEXT('USERENV','CURRENT_USER') FROM DUAL;
```

All users

```
-- Every user in the database (needs DBA privileges)
SELECT username, account_status, created FROM dba_users ORDER BY created
DESC;

-- Users visible to you (no special privileges needed)
SELECT username FROM all_users;

-- Only your own account details
SELECT * FROM user_users;
```

Hide Oracle's built-in accounts (12c and later)

```
SELECT username, account_status
FROM dba_users
WHERE oracle_maintained = 'N';
```

Other useful checks

```
SELECT username, default_tablespace, temporary_tablespace FROM dba_users;
SELECT * FROM dba_ts_quotas WHERE username = 'AASIF';
```

4. Switch User

Oracle has no `SWITCH USER` command. You **reconnect** as the other user.

```
CONNECT aasif/StrongPass#123
CONN aasif                 -- prompts for the password (safer)
CONN aasif@XEPDB1          -- connect to a specific service
CONN / AS SYSDBA           -- OS authentication as SYSDBA
CONN sys@XEPDB1 AS SYSDBA  -- SYS must always use AS SYSDBA
```

- A new `CONNECT` ends your current session and starts a new one.
- Passwords containing special characters may need double quotes: `CONN aasif/"Pass#123"`.

Related: switch schema without switching user

```
ALTER SESSION SET CURRENT_SCHEMA = hr;
```

This lets you query `hr`'s tables without typing `hr.` each time. It only changes name resolution; you do not gain `hr`'s privileges.

5. Change Password

As a DBA (any user)

```
ALTER USER aasif IDENTIFIED BY "NewPass#456";
```

As the user themselves (in SQL*Plus, hides the typing)

```
PASSWORD
-- or
PASSWORD aasif
```

Related account tasks

```
ALTER USER aasif PASSWORD EXPIRE;     -- force change at next login
ALTER USER aasif ACCOUNT UNLOCK;      -- unlock after failed attempts
ALTER USER aasif ACCOUNT LOCK;
ALTER USER aasif QUOTA 100M ON users; -- change quota
```

6. Grant Permissions

Syntax

```
GRANT privilege [, privilege ...] TO user_or_role;
GRANT privilege ON object TO user_or_role;
```

Examples

```
-- System privileges (no ON clause)
GRANT CREATE SESSION, CREATE TABLE, CREATE VIEW TO aasif;

-- Object privileges (ON the object)
GRANT SELECT, INSERT, UPDATE ON hr.employees TO aasif;
GRANT EXECUTE ON hr.my_procedure TO aasif;
GRANT ALL PRIVILEGES ON hr.employees TO aasif;   -- all object privileges
```

Take permissions back

```
REVOKE CREATE VIEW FROM aasif;
REVOKE INSERT ON hr.employees FROM aasif;
```

Grant options (passing permission on to others)

Option	Applies to	Meaning
`WITH ADMIN OPTION`	System privileges and roles	The grantee can grant the same privilege or role to others. Revoking from the grantee does **not** cascade.

Option	Applies to	Meaning
`WITH GRANT OPTION`	Object privileges	The grantee can grant the same object privilege to others. Revoking **does** cascade.

```
GRANT CREATE TABLE TO aasif WITH ADMIN OPTION;
GRANT SELECT ON hr.employees TO aasif WITH GRANT OPTION;
```

Check what a user has

```
SELECT * FROM dba_sys_privs  WHERE grantee = 'AASIF';  -- system privileges
SELECT * FROM dba_tab_privs  WHERE grantee = 'AASIF';  -- object privileges
SELECT * FROM dba_role_privs WHERE grantee = 'AASIF';  -- roles

-- Your own (no DBA needed)
SELECT * FROM user_sys_privs;
SELECT * FROM user_tab_privs;
SELECT * FROM user_role_privs;
SELECT * FROM session_privs;   -- everything active right now
```

7. Types of Permissions (Privileges)

Oracle has two main types.

A. System Privileges

Right to perform an action on the database, or on a class of objects. Not tied to one specific object.

Privilege	Allows
`CREATE SESSION`	Log in to the database
`CREATE TABLE`	Create tables in own schema
`CREATE VIEW`	Create views in own schema
`CREATE SEQUENCE`	Create sequences
`CREATE PROCEDURE`	Create procedures, functions, packages
`CREATE TRIGGER`	Create triggers

Privilege	Allows
`CREATE USER`	Create users
`ALTER USER`	Modify users (including passwords)
`DROP USER`	Drop users
`CREATE ANY TABLE`	Create tables in **any** schema
`SELECT ANY TABLE`	Query **any** table in the database
`UNLIMITED TABLESPACE`	Use unlimited space on all tablespaces
`ALTER SYSTEM`	Change instance-level settings

Note: the `ANY` privileges are powerful. Grant them carefully.

B. Object Privileges

Right to perform an action on one **specific object** (table, view, procedure, sequence).

Privilege	Allows	Applies to
`SELECT`	Read data	Table, view, sequence
`INSERT`	Add rows	Table, view
`UPDATE`	Modify rows (can be limited to columns)	Table, view
`DELETE`	Remove rows	Table, view
`ALTER`	Change the structure	Table, sequence
`INDEX`	Create an index	Table
`REFERENCES`	Create a foreign key to it	Table
`EXECUTE`	Run it	Procedure, function, package
`ALL`	Every privilege valid for that object	Any

```
-- Column-level UPDATE
GRANT UPDATE (salary) ON hr.employees TO aasif;
```

Quick comparison

	System privilege	Object privilege
Scope	Database-wide action	One specific object
Syntax	`GRANT priv TO user`	`GRANT priv ON obj TO user`
Example	`CREATE TABLE`	`SELECT ON hr.employees`
Dictionary view	`DBA_SYS_PRIVS`	`DBA_TAB_PRIVS`

8. Roles

A **role** is a named bundle of privileges. Instead of granting ten privileges to each user, grant the role once.

Common predefined roles

Role	What it provides
`CONNECT`	Basically `CREATE SESSION` only in 12c and later (older versions bundled more)
`RESOURCE`	Create own tables, sequences, procedures, triggers, types, indexes, clusters
`DBA`	Almost all system privileges. Full administration
`SELECT_CATALOG_ROLE`	Read access to data dictionary views (`DBA_*`)
`EXECUTE_CATALOG_ROLE`	Execute data dictionary packages
`EXP_FULL_DATABASE` / `IMP_FULL_DATABASE`	Full export and import
`PUBLIC`	Special: every user automatically has it

See all roles in your database:

```
SELECT role FROM dba_roles ORDER BY role;
```

Assign a role to a user

```
GRANT CONNECT, RESOURCE TO aasif;      -- typical development setup
GRANT DBA TO aasif;                    -- full admin (use sparingly)
GRANT SELECT_CATALOG_ROLE TO aasif;
```

Remove it:

```
REVOKE RESOURCE FROM aasif;
```

Create your own (custom) role

```
-- 1. Create the role
CREATE ROLE report_reader;

-- 2. Put privileges in it
GRANT CREATE SESSION TO report_reader;
GRANT SELECT ON hr.employees   TO report_reader;
GRANT SELECT ON hr.departments TO report_reader;

-- 3. Give the role to users
GRANT report_reader TO aasif;
GRANT report_reader TO sara;
```

Change the role once and every user holding it is updated.

```
DROP ROLE report_reader;
```

Inspect roles

```
SELECT * FROM dba_role_privs WHERE grantee = 'AASIF';        -- roles a user
has
SELECT * FROM role_sys_privs WHERE role = 'RESOURCE';        -- system
privileges in a role
SELECT * FROM role_tab_privs WHERE role = 'REPORT_READER';   -- object
privileges in a role
```

Role caveats

- Privileges received **through a role** do not work inside named PL/SQL procedures (definer's rights). Grant such privileges **directly** to the user instead.

- `CONNECT, RESOURCE` is fine for learning and practice. For production, prefer custom roles with only the privileges needed (**principle of least privilege**).
- Use `QUOTA` explicitly. Do not rely on a role to supply storage space.

9. Complete Workflow Example

```
-- As SYSTEM or a DBA, inside a PDB
ALTER SESSION SET CONTAINER = XEPDB1;

CREATE USER student1 IDENTIFIED BY "Student#123"
  DEFAULT TABLESPACE users
  TEMPORARY TABLESPACE temp
  QUOTA 100M ON users;

GRANT CONNECT, RESOURCE TO student1;
GRANT CREATE VIEW TO student1;

-- Switch and verify
CONN student1/"Student#123"@XEPDB1
SHOW USER;
SELECT * FROM session_privs;

-- Change password
PASSWORD
```

10. Quick Reference

Task	Command
Create user	`CREATE USER u IDENTIFIED BY p;`
Show current user	`SHOW USER;`
List all users	`SELECT username FROM dba_users;`
Switch user	`CONN u/p`
Change password	`ALTER USER u IDENTIFIED BY newp;`
Lock / unlock	`ALTER USER u ACCOUNT LOCK / UNLOCK;`

Task	Command
Grant system privilege	`GRANT CREATE TABLE TO u;`
Grant object privilege	`GRANT SELECT ON t TO u;`
Revoke	`REVOKE priv FROM u;`
Grant role	`GRANT CONNECT, RESOURCE TO u;`
Create role	`CREATE ROLE r;`
Drop user	`DROP USER u CASCADE;`